

Internship Learning Roadmap

Duration: 12 Weeks | Domain: Backend Development + AI/LLM Integration

Goal: Progress from scratch to building production-ready AI-powered applications.

Program Overview

Phase	Weeks	Focus Area
Phase 1	Weeks 1–4	Backend Foundations — Python, FastAPI, Django
Phase 2	Weeks 5–6	Databases & ORM
Phase 3	Weeks 7–8	Advanced Django & System Design
Phase 4	Weeks 9–12	LLMs & LangChain Integration

Evaluation Criteria

Criteria	Description
Technical Understanding	Weekly code reviews and short concept checks
Problem-Solving Ability	Approach to debugging and architectural decisions
Code Quality	Clean, readable, and well-structured code
Timely Delivery	Consistent progress and meeting task deadlines
Communication	Raising blockers early and maintaining a learning mindset

Recommended Tech Stack — Overview

The following tools and technologies are used across the program. Interns are not expected to know these upfront — each will be introduced progressively within the relevant phase.

Category	Tools / Technologies
Language	Python 3.11+
Frameworks	FastAPI, Django, Django REST Framework (DRF)
Database	PostgreSQL, SQLite (for local dev), MySQL (basics)
ORM	SQLAlchemy, Django ORM
Auth	JWT (via python-jose / djangorestframework-simplejwt)
Testing	pytest, pytest-django, Postman
DevOps	Docker, Docker Compose

Task Queue	Celery + Redis
Caching	Redis, Memcached
Version Control	Git & GitHub (branching, PRs, commit hygiene)
AI / LLM	OpenAI API, Ollama, LangChain, FAISS, ChromaDB
Frontend (Bonus)	React or plain HTML/JS for demo UI

Phase 1: Backend Foundations

Weeks 1–4

Goal: Build comfort with Python and modern backend frameworks. By the end of this phase, interns should be able to create, secure, and containerise a REST API.

Tech Stack — Phase 1

Python 3.11+	FastAPI	Pydantic	SQLAlch emy	JWT	Docker	pytest	Git & GitHub
-----------------	---------	----------	----------------	-----	--------	--------	-----------------

Week 1 — Python Essentials

- OOP: classes, inheritance, magic methods
- Virtual environments (venv) and package management (pip)
- Working with APIs: requests, json
- File I/O and exception handling
- Git basics: init, commit, push, branching, pull requests

Weeks 2–3 — FastAPI

- Setting up a REST API project from scratch
- Routing, path parameters, and query params
- Request/Response models using Pydantic
- Dependency Injection
- JWT-based authentication — securing endpoints
- Connecting FastAPI to a database using SQLAlchemy
- Dockerising a FastAPI app — Dockerfile and docker-compose basics
- Testing APIs — Postman and pytest

Week 4 — Django Basics

- Project vs App structure
- MVT Architecture — Models, Views, Templates
- ORM basics and migrations
- Django Admin panel
- Forms and URL routing

End-of-Phase 1 — Project Options

Intern picks one of the following:

#	Project	What They Build
1	Personal Portfolio API	FastAPI backend serving portfolio data — Projects, Skills, Experience — with JWT auth, CRUD, SQLAlchemy, Docker, and auto-docs at /docs.

2	URL Shortener API	Create and resolve short URLs with click tracking, expiry support, user accounts, and a stats endpoint. FastAPI + SQLAlchemy + Docker.
3	Weather Journal API	Log personal weather observations with location tagging. Integrates a public weather API, stores history in SQLite/PostgreSQL, supports filtering by date/location.

All options require: full CRUD, Pydantic validation, JWT auth, Dockerised setup, /docs, and at least 5 pytest cases. Deliverable: GitHub repo with README covering setup and endpoints.

Phase 2: Databases & ORM

Weeks 5–6

Goal: Gain hands-on experience with relational databases, write efficient queries, and model real-world data relationships.

Tech Stack — Phase 2

PostgreSQL	MySQL	SQLAlchemy	Django ORM	Git & GitHub
------------	-------	------------	------------	--------------

Week 5 — SQL Fundamentals

- CRUD operations — SELECT, INSERT, UPDATE, DELETE
- Joins — INNER, LEFT, RIGHT
- Database Normalisation — 1NF, 2NF, 3NF
- Indexes and query performance basics

Week 6 — MySQL, PostgreSQL & ORM Practice

- Database setup and CLI usage for MySQL and PostgreSQL
- Connecting Python applications to databases
- SQLAlchemy — models, sessions, relationships
- Django ORM — advanced queries, `select_related`, `prefetch_related`, aggregations

End-of-Phase 2 — Project Options

Intern picks one of the following:

#	Project	What They Build
1	Task Manager API	RESTful task management with user accounts, task priorities, due dates, status filters, and a raw SQL report endpoint for overdue tasks per user.
2	Expense Tracker API	Track personal expenses by category and date. Supports monthly summaries, category-wise breakdowns, and a raw SQL report for top spending categories.
3	Library Management System	Manage books, members, and borrow records. Track availability, overdue returns, and generate a report of most borrowed books using raw SQL.

All options require: PostgreSQL, at least 2 table relationships, user auth, CRUD with filtering/sorting, and one raw SQL report endpoint. Deliverable: local demo with Postman collection or Swagger UI.

Phase 3: Advanced Django & System Design

Weeks 7–8

Goal: Move into production-grade Django patterns — async tasks, caching, permissions, and multi-service Docker setups.

Tech Stack — Phase 3

Django	DRF	Celery	Redis	PostgreS QL	Docker Compose	pytest-dj ango	Git & GitHub
--------	-----	--------	-------	-------------	----------------	----------------	--------------

Week 7 — Django REST Framework & Middleware

- DRF — ViewSets, Serializers, Routers
- Middleware — request lifecycle, custom middleware
- Django Signals — post_save, pre_delete
- Authentication & Permissions in DRF — IsAuthenticated, custom permission classes
- Caching with Redis / Memcached basics

Week 8 — Async, Testing & Docker Compose

- Async views in Django
- Background tasks with Celery + Redis
- Unit testing — pytest-django, mocking, test factories
- Project structuring best practices — apps, services, utilities
- Docker Compose for multi-service setup: Django + PostgreSQL + Redis

End-of-Phase 3 — Project Options

Intern picks one of the following:

#	Project	What They Build
1	Multi-User Blog Platform	Posts, Categories, Comments with role-based access (Admin, Author, Reader). Redis caching on popular posts, Celery email on new comment, full Docker Compose stack.
2	Event Management System	Create and manage events with RSVPs and attendee tracking. Role-based access, Redis caching on upcoming events, Celery reminder notifications, full Docker Compose.
3	Ticket Support System	Users raise tickets, agents respond and update status. Priority queues, SLA tracking, Celery for escalation alerts, Redis caching, role-based DRF permissions.

All options require: DRF with pagination and filtering, Redis caching, at least one Celery task, full Docker Compose setup, and at least 10 test cases. Deliverable: GitHub repo with Docker setup and walkthrough doc.

Phase 4: LLMs & LangChain Integration

Weeks 9–12

Goal: Understand how LLMs work and build AI-powered backend applications using LangChain and FastAPI.

Tech Stack — Phase 4

Open AI API	Ollama	LangChain	FAISS	ChromaDB	FastAPI	PostgreSQL	Redis	Docker	Git & GitHub
-------------	--------	-----------	-------	----------	---------	------------	-------	--------	--------------

Weeks 9–10 — LLM Fundamentals & LangChain Basics

- Introduction to LLMs — OpenAI API, Ollama (local models)
- Prompt engineering — zero-shot, few-shot, chain-of-thought
- LangChain fundamentals — chains, prompts, memory
- RAG pipeline: document loaders, text splitters
- Embedding models and vector stores (FAISS / ChromaDB)
- Retrieval chains for document Q&A;

Weeks 11–12 — Building AI-Powered Backends

- LangChain tool integrations — web search, database, custom tools
- Stateful conversations with LangChain memory
- Building a chatbot backend using FastAPI + LangChain
- Streaming responses via FastAPI
- Connecting backend to a simple frontend (HTML/JS or React)

Final Capstone — Project Options

Intern picks one of the following:

#	Project	What They Build
1	AI-Powered Knowledge Assistant	Upload PDFs/text files, query them via a RAG pipeline, maintain per-user session memory. FastAPI + LangChain + PostgreSQL + vector store, fully Dockerised.
2	AI Resume Reviewer	Upload a resume and job description, get AI-generated feedback, match score, and improvement suggestions. Stores history per user. FastAPI + LangChain + PostgreSQL.
3	Study Assistant Chatbot	Upload study material, ask questions, get context-aware answers with follow-up support. Multi-turn memory per session, streaming responses, FastAPI + LangChain + ChromaDB.

All options require: JWT auth, document upload endpoints, RAG pipeline, multi-turn memory, PostgreSQL for history, fully Dockerised stack, and .env config. Deliverable: GitHub repo + 3–5 min demo video + README

with architecture diagram.

Internship Learning Roadmap — Backend Development + AI/LLM Integration — 12 Weeks